

Kota 2: Embodied Agent Swarms via Recursive Assembly

Erick Oduniyi
Intelligent Interfaces Group
eeoduniyi@gmail.com

August 5, 2026

Abstract

Building upon the foundational architecture of the Kota agent—a local-first autonomous agent engineered around an event-driven Rust core—we present Kota 2, a theoretical framework for physically grounded, community-aware agent swarms. The central contribution is the formalization of *cognitive voxels*: discrete, AST-validated software primitives that mirror the digital materials paradigm from recursive robotic assembly. We show that this discretization enables three formal properties: (i) exponential task scaling through hierarchical agent delegation, (ii) a quantitative creativity metric derived from Assembly Theory, and (iii) mass-conserving autopoietic skill evolution inspired by Flow-Lenia dynamics. We further introduce polycentric governance protocols based on Ostrom’s commons principles and cooperative Shapley allocation for energy-just multi-operator scheduling. Finally, we specify the interface layer that extends agent actuation from digital tool use to physical hardware control via the Model Context Protocol.

1 Introduction

The transition from single-agent LLM wrappers to multi-agent swarms requires fundamental shifts in memory, orchestration, and security. Kota 1 demonstrated the viability of treating an agent as a *software circuit*—an event-driven Rust core with real-time hardware sensing, hierarchical memory via Turso/libSQL, and modular tool composition through the *Option Keyboard* [Abdel-Rahman et al., 2022]. However, it remained bound to the paradigm of the isolated digital assistant.

Kota 2 scales this circuitry beyond the digital sandbox. The central argument is that an autonomous agent must operate as an active participant in shared physical and digital ecosystems. We organize this extension along seven theoretical axes:

1. **Software foundations** (Section 2): Graph RAG memory, ADK orchestration, MIMO security.
2. **Cognitive voxels** (Section 3): The central thesis—discretizing cognition via digital materials.
3. **Assembly metrics** (Section 4): Quantifying cognitive utility via Assembly Theory.
4. **Polycentric governance** (Section 5): Commons-based swarm coordination.
5. **Energy justice** (Section 6): Shapley allocation and power-aware scheduling.
6. **Autopoiesis** (Section 7): Mass-conserving, self-evolving skill trees.
7. **Physical interface** (Section 8): Bridging digital cognition to real-world actuation.

2 Software Architecture Foundations

2.1 Spectral Memory via Graph RAG

Kota 1’s memory relied on sparse keyword retrieval (LIKE %keyword%) over a local SQLite store. Kota 2 augments this with two complementary backends:

- **Vertex AI Memory Bank:** Long-horizon episodic persistence with semantic embedding retrieval.
- **Cloud Spanner Graph:** Structural relationship mapping across conversational context, codebase entities, and the agent’s knowledge wiki.

Together, these form a *spatial Graph RAG* framework. We define spectral retrieval as the decomposition of the memory adjacency matrix $\mathbf{A} \in \mathbb{R}^{n \times n}$ via truncated SVD:

$$\mathbf{A} \approx \mathbf{U}_k \mathbf{\Sigma}_k \mathbf{V}_k^\top, \quad k \ll n \quad (1)$$

where k is the rank of the approximation. Queries are projected into the k -dimensional spectral space, enabling retrieval by semantic proximity rather than lexical overlap.

2.2 Multi-Agent Orchestration via ADK

We decompose the monolithic event loop into specialized, concurrent agents using Google’s Agent Development Kit (ADK):

- **Sensing Agent $\mathcal{A}_s$:** Polls hardware state—CPU/GPU temperature, wattage, bandwidth (rxkbps, txkbps)—and enforces Neotrickle bandwidth shaping.
- **Execution Agent $\mathcal{A}_e$:** Writes code and invokes tools via the Option Keyboard.
- **Critic Agent $\mathcal{A}_c$:** Evaluates outputs, filters evaluation noise (overly strict tests, low-coverage assertions), and enforces security policies.

2.3 Cognitive RF Security via MIMO Prompting

Adversarial prompt injection is structurally analogous to RF interference in wireless communication systems. We formalize this via *MIMO Prompting*:

Definition 1 (MIMO Prompting). *Given a prompt $\mathbf{p}$ and an ensemble of M sub-agents $\{\mathcal{A}_1, \dots, \mathcal{A}_M\}$ operating at temperatures $\{\tau_1, \dots, \tau_M\}$, the beamformed output is:*

$$\hat{\mathbf{y}} = \sum_{i=1}^M w_i \cdot \mathcal{A}_i(\mathbf{p}; \tau_i), \quad \text{where } w_i = \frac{\text{coh}(\mathcal{A}_i, \bar{\mathcal{A}})}{\sum_j \text{coh}(\mathcal{A}_j, \bar{\mathcal{A}})} \quad (2)$$

where $\text{coh}(\cdot, \cdot)$ measures semantic coherence against the ensemble mean $\bar{\mathcal{A}}$. Adversarial injections produce low-coherence outliers that are suppressed by the weighting scheme.

3 Cognitive Voxels

This section presents the central thesis of Kota 2.

The MIT Center for Bits and Atoms reconceptualizes manufacturing through *digital materials*: standardized primitive building blocks (functional voxels) that vastly simplify error correction and enable recursive self-assembly [Abdel-Rahman et al., 2022, Smith et al., 2025]. Micro-scale robots assemble mid-scale robots, which in turn assemble macro-structures—achieving exponential scaling through identical, reusable primitives.

Algorithm 1 Pre-Execution Load Simulation

Require: Task T , delegation tree $\mathcal{T}$, energy budget $E_{\max}$

Ensure: Approved execution plan or veto

```
1:  $\hat{E} \leftarrow 0$  ▷ Estimated total energy
2: for each leaf node  $n_i \in \text{leaves}(\mathcal{T})$  do
3:    $\hat{E} \leftarrow \hat{E} + c(n_i) \cdot \text{depth}(n_i)$ 
4: end for
5:  $\hat{M} \leftarrow \sum_{n_i \in V} \text{ctx\_tokens}(n_i)$  ▷ Total memory
6: if  $\hat{E} > E_{\max}$  or  $\hat{M} > M_{\max}$  then
7:   return Veto("Exceeds energy/memory budget")
8: else
9:   return Approve( $\mathcal{T}$ )
10: end if
```

Definition 2 (Cognitive Voxel). *A cognitive voxel $v \in \mathcal{V}$ is an atomic, AST-validated software action with the following properties:*

1. **Determinism:** *Given identical inputs, v produces identical outputs.*
2. **Boundedness:** *The execution cost $c(v)$ is upper-bounded: $c(v) \leq c_{\max}$.*
3. **Composability:** *Voxels compose via sequential $(v_1 \circ v_2)$ or parallel $(v_1 \parallel v_2)$ operators.*
4. **Verifiability:** *The postcondition of v is machine-checkable (e.g., AST validity, type safety).*

Examples of cognitive voxels include: an AST-validated function write, a cryptographically verified API call, a sandboxed shell execution, and a typed file read. The agent *cannot* act continuously; it is structurally constrained to compose only pre-validated blocks.

3.1 Recursive Task Delegation

The self-replication demonstrated by hierarchical modular swarms [Abdel-Rahman et al., 2022] maps directly onto Kota’s `delegate_task` primitive. We formalize the recursive delegation tree:

Definition 3 (Delegation Tree). *A delegation tree $\mathcal{T} = (V, E)$ is a rooted tree where each node $n_i \in V$ is an agent instance and each edge $(n_i, n_j) \in E$ represents a task delegation. The root agent n_0 delegates to children $\{n_1, \dots, n_k\}$, which may recursively delegate further.*

Property 1 (Exponential Scaling). *For a delegation tree of depth d with branching factor b , the total parallelism is $O(b^d)$ while the error containment per leaf is $O(1/b^d)$, since each leaf operates on an isolated, bounded subtask.*

Before execution, the root agent simulates the cognitive load, memory consumption, and energy cost of the planned assembly tree (Algorithm 1). If simulation indicates high probability of failure or thermal throttling, the plan is vetoed and returned to the planning layer.

4 Assembly Theory as a Creativity Metric

Assembly Theory quantifies the complexity of objects based on their causal formation histories [Sharma et al., 2023]. The *assembly index* $a(x)$ is the minimum number of joining operations required to construct object x from basic building blocks, where previously constructed sub-assemblies may be reused as single components.

Definition 4 (Assembly Index for Tool Chains). Let $\mathcal{V} = \{v_1, \dots, v_n\}$ be the set of cognitive voxels. For a composite task $T = v_{i_1} \circ v_{i_2} \circ \dots \circ v_{i_m}$, the assembly index is:

$$a(T) = \min_{\pi} |\pi| \quad \text{s.t. } \pi \text{ produces } T \text{ from } \mathcal{V} \text{ with sub-assembly reuse} \quad (3)$$

where π is a construction pathway and $|\pi|$ counts irreducible joining steps.

Definition 5 (Copy Number Threshold). Let $N(T)$ denote the number of times task pattern T appears across distinct operational contexts. The selection signature is the pair $(a(T), N(T))$. We classify:

$$\text{class}(T) = \begin{cases} \text{Verified-Macro} & \text{if } a(T) > \alpha \text{ and } N(T) > \beta \\ \text{Memorized-Template} & \text{if } a(T) \leq \alpha \text{ and } N(T) > \beta \\ \text{Novel-Exploration} & \text{if } a(T) > \alpha \text{ and } N(T) \leq \beta \\ \text{Noise} & \text{otherwise} \end{cases} \quad (4)$$

where α and β are domain-specific thresholds [Seet, 2025].

Tasks classified as **Verified-Macro** are automatically promoted into the agent’s skill tree as callable single-step tools, compressing future assembly paths. Tasks classified as **Novel-Exploration** with high utility are candidates for creative reward by the Critic Agent $\mathcal{A}_c$.

5 Polycentric Governance

Scaling to multi-operator environments transforms local compute into a *Common-Pool Resource* (CPR). GPU VRAM, network bandwidth, and SQLite I/O are rival and partially excludable—the defining characteristics of a CPR [Ostrom, 1990].

We embed Ostrom’s eight design principles as algorithmic constraints:

Ostrom Principle	Kota 2 Implementation
Defined boundaries	Cryptographic isolation of per-agent SQLite memory and VRAM quotas
Local congruence	Agent capabilities bounded by real-time hardware measurements from $\mathcal{A}_s$
Collective choice	Sub-agents vote via MIMO synthesis (Eq. 2) before state-altering commits
Monitoring	$\mathcal{A}_c$ continuously audits resource consumption per agent
Graduated sanctions	Bandwidth throttling proportional to context-window overuse
Conflict resolution	Shapley-value arbitration (Section 6)

Table 1: Mapping Ostrom’s commons principles to swarm governance.

5.1 Stigmergic Coordination

Rather than explicit $O(n^2)$ message passing between agents, coordination proceeds *stigmergically* through the shared Graph RAG store. Agents annotate memory nodes with metadata tags—utility scores, temporal relevance, and access frequency—that implicitly guide the retrieval paths of other agents [Minsky, 1986]. This mirrors biological stigmergy in ant colonies and scales sub-linearly with swarm size.

Research on real-time human-AI deliberation further motivates structural pacing interventions—artificial words-per-minute throttling and cohort transfers—to prevent agents from overwhelming human participants [Qian, 2025].

6 Energy Justice and Shapley Allocation

Power in agent systems has dual meaning: thermodynamic energy (watts, joules) and socio-economic leverage. Deep learning training carries substantial carbon costs [Strubell et al., 2019], framing energy consumption as an environmental justice concern.

6.1 Energy Score as Active Constraint

Kota 1 introduced the Energy Score $\mathcal{E}$ as a passive metric. Kota 2 elevates it to an *active constraint*:

$$\mathcal{E}(T) = \int_0^{\Delta t} P(t) dt \leq E_{\max} \quad (5)$$

where $P(t)$ is instantaneous power draw and $E_{\max}$ is the per-task energy budget. Tasks exceeding $E_{\max}$ trigger automatic decomposition into lower-energy sub-tasks via the delegation tree (Section 3).

6.2 Cooperative Resource Allocation

When K operators share a compute cluster, we allocate resources via Shapley values from cooperative game theory. Let $v : 2^K \rightarrow \mathbb{R}$ be the characteristic function mapping operator coalitions to utility. The Shapley value for operator i is:

$$\phi_i(v) = \sum_{S \subseteq K \setminus \{i\}} \frac{|S|!(|K| - |S| - 1)!}{|K|!} [v(S \cup \{i\}) - v(S)] \quad (6)$$

GPU time is allocated proportionally to ϕ_i , ensuring that no single operator monopolizes hardware and that each receives compute proportional to their marginal contribution to community utility.

6.3 Legibility Pauses

To counteract the structural power asymmetry between agent velocity and human comprehension [Anonymous, 2026b], the system enforces *mandatory legibility pauses*: for any action with estimated impact $I(a) > I_{\text{threshold}}$, the agent halts execution and requests a human verification token before proceeding. This tethers operational velocity to the operator’s cognitive bandwidth.

7 Autopoietic Skill Evolution

Artificial life simulations—particularly Flow-Lenia [Plantec et al., 2023]—demonstrate that complex, self-organizing behavior emerges from simple localized rules under two critical constraints: *mass conservation* and *parameter localization*.

7.1 Token Conservation

We map Flow-Lenia’s mass conservation directly onto the agent’s finite token budget. Let $\mathcal{M}$ denote the total “cognitive mass” (active context tokens plus stored memory entries). We enforce:

$$|\mathcal{M}(t)| = \text{const.} \quad \forall t \quad (7)$$

If a new skill s_{new} is activated, requiring δ tokens, an equivalent mass of low-utility memory must be decayed:

$$\mathcal{M}(t+1) = \mathcal{M}(t) \cup \{s_{\text{new}}\} \setminus \arg \min_{\substack{S \subseteq \mathcal{M}(t) \\ |S|=\delta}} \sum_{s \in S} u(s) \quad (8)$$

where $u(s)$ is the utility score of memory entry s , computed from access frequency and recency.

7.2 Self-Evolving Skill Kernels

Rather than a static tool array, the skill tree operates as a continuous kernel function. Each tool v_i has a selection weight:

$$w_i(t) = \frac{\exp(\eta \cdot r_i(t)/c_i(t))}{\sum_j \exp(\eta \cdot r_j(t)/c_j(t))} \quad (9)$$

where $r_i(t)$ is the cumulative success rate of tool v_i up to time t , $c_i(t)$ is its average energy cost, and η is a temperature parameter. Tools with high reward-to-cost ratios are preferentially selected. Over time, the agent dynamically rewrites its own tool definitions—composing high-weight primitives into new compound tools—achieving *software autopoiesis* [Anonymous, 2026a].

8 The Physical Interface Layer

The ultimate trajectory of agentic technology is physical actuation [Liang et al., 2023, Suchman, 1987]. Existing paradigms demonstrate that language models can map natural language to robotic skill vocabularies, outputting executable control scripts. The open-source Model Context Protocol (MCP) has been extended to IoT devices and local microcontrollers, providing a standardized JSON-RPC interface for reading sensors and triggering actuators without cloud dependency.

8.1 Asymmetric Energy Budgets

Physical actions carry fundamentally different risk profiles than digital ones. Writing a file consumes microwatts; operating a CNC spindle consumes kilowatts. We formalize this via an *asymmetric energy budget*:

$$E_{\text{max}}^{\text{phys}} = \gamma \cdot E_{\text{max}}^{\text{dig}}, \quad \gamma \gg 1 \quad (10)$$

Actions tagged as physical actuation require both a higher Shapley-value justification (Eq. 6) and mandatory human-in-the-loop verification, serving as the robotic equivalent of a software sandbox.

8.2 Tool-Use Cascades

Situated cognition holds that intelligence arises from continuous interaction with an environment [Suchman, 1987]. In physical fabrication, this manifests as *tool-use cascades*: mastery of one tool enables construction of a more advanced tool, yielding exponential fabrication capability from minimal starting resources.

Kota 2 is designed to recognize and optimize these cascades. The agent assists the operator not merely in fabricating a singular end-use part, but in designing the automated jigs, fixtures, and secondary machines that accelerate all future physical workflows—bootstrapping the operator’s fabrication infrastructure recursively.

9 Conclusion

We have presented Kota 2, a theoretical framework that extends the autonomous coding agent from a digital tool to a physically grounded, community-aware system. The key contributions are:

1. **Cognitive voxels** (Definition 1): Discretizing agent actions into AST-validated primitives, enabling recursive delegation with exponential scaling (Property 1).
2. **Assembly-theoretic creativity metrics** (Eq. 3–4): Quantifying cognitive utility and autonomously evolving the skill tree.
3. **Polycentric governance** (Table 1): Embedding Ostrom’s commons principles as algorithmic constraints for multi-operator swarms.
4. **Energy-just allocation** (Eq. 5–6): Shapley-value resource distribution with mandatory legibility pauses.
5. **Autopoietic skill evolution** (Eq. 7–9): Mass-conserving, self-evolving tool selection inspired by Flow-Lenia.
6. **Physical interface specification** (Eq. 10): Asymmetric energy budgets and tool-use cascades for real-world actuation.

Future work includes empirical validation of the assembly index on production codebases, deployment of the Shapley allocator on multi-GPU clusters, and integration with robotic assembly platforms for physical fabrication tasks.

References

- Amira Abdel-Rahman, Christopher Cameron, Benjamin Jenett, Miana Smith, and Neil Gershenfeld. Self-replicating hierarchical modular robotic swarms. *Communications Engineering*, 1(1):35, 2022.
- Anonymous. Autopoiesis: A self-evolving system paradigm for llm serving under runtime dynamics. *arXiv preprint arXiv:2604.07144*, 2026a.
- Anonymous. The great inversion: Precedent, power, and moral reciprocity in human-ai relations. *AI Futures Policy Review*, 2026b.
- Jacky Liang et al. Code as policies: Language model programs for embodied control. *IEEE International Conference on Robotics and Automation (ICRA)*, 2023.
- Marvin Minsky. *The Society of Mind*. Simon and Schuster, 1986.
- Elinor Ostrom. *Governing the Commons: The Evolution of Institutions for Collective Action*. Cambridge University Press, 1990.
- Erwan Plantec, Gautier Hamon, Mayalen Etcheverry, Pierre-Yves Oudeyer, Clément Moulin-Frier, and Bert Wang-Chak Chan. Flow-lenia: Towards open-ended evolution in cellular automata through mass conservation and parameter localization. In *Artificial Life Conference Proceedings 35*, page 131. MIT Press, 2023.
- et al. Qian. Deliberate lab: A platform for real-time human-ai social experiments. *arXiv preprint arXiv:2510.13011*, 2025.

- et al. Seet. assembly-theory: A rust library for molecular assembly index calculation. *Journal of Open Source Software*, 2025.
- Abhishek Sharma, Dániel Czégel, Michael Lachmann, Christopher P Kempes, Sara I Walker, and Leroy Cronin. Assembly theory explains and quantifies selection and evolution. *Nature*, 622(7982):321–328, 2023.
- Miana Smith, Jack Forman, Amira Abdel-Rahman, Sophia Wang, and Neil Gershenfeld. Voxel invention kit: Reconfigurable building blocks for prototyping interactive electronic structures. In *CHI Conference on Human Factors in Computing Systems*, 2025.
- Emma Strubell, Ananya Ganesh, and Andrew McCallum. Energy and policy considerations for deep learning in nlp. *arXiv preprint arXiv:1906.02243*, 2019.
- Lucy A Suchman. *Plans and situated actions: The problem of human-machine communication*. Cambridge university press, 1987.